

模拟卷 05-移动支付积分系统题目与详解

目录

1	软件工程与计算 II 期末模拟卷 05：移动支付积分系统（题目与详解）	2
2	A 卷题目	2
3	B 详解	4
4	C 复盘清单	9

1 软件工程与计算 II 期末模拟卷 05：移动支付积分系统 (题目与详解)

1.1 卷面说明

本卷按 2020、2022 的完整十题结构设计，并纳入 2025 的 Spring Boot 三层代码、策略模式、代码改造、测试与 HCI 题型。总分 100 分，每题 10 分。

场景：移动支付积分系统提供扫码付款与积分计算。主要参与者包括消费者、商家、支付平台、银行系统、积分系统。系统采用 Web 架构，客户端为浏览器，服务器端采用 Spring Boot，数据持久化由 Repository 完成。

2 A 卷题目

2.1 一、简答题（10 分）

1. 软件工程。
2. 演化模型及优缺点。
3. 正向工程、逆向工程、再工程。

2.2 二、需求题（10 分）

围绕 扫码付款与积分计算：

1. 写出 2 个用例，说明参与者和主成功场景。
2. 写出一个业务需求、一个用户需求和一个系统级需求。
3. 判断“系统需要向外部系统提供 API 供调用”属于哪类需求，并说明原因。

2.3 三、需求分析题（10 分）

根据场景，完成 扫码付款与积分计算的概念类图：

1. 识别问题域概念类。
2. 写出概念类之间的关联、聚合、组合或继承。
3. 写出关键属性并画出类图。

2.4 四、体系结构设计题（10 分）

系统采用 Web 分层架构：

1. 画出物理包图，要求体现客户端 `html/css/javascript/presentation`，服务器端 `Controller`、`Service`、`Repository`、`Domain`、`Database`，并标注 HTTP、REST API、JSON。
2. 针对 扫码付款与积分计算写出 `Service` 接口、`Repository` 方法声明、`Controller` 调用 `Service` 的依赖注入代码片段，写出包名。

2.5 五、详细设计题（10 分）

针对 扫码付款与积分计算：

1. 画出 `Controller`、`Service`、`Repository`、外部接口之间的顺序图。
2. 简述集中式、分散式、委托式控制策略。
3. 判断本场景适合的控制策略并说明理由。

2.6 六、模块化与信息隐藏题（10 分）

系统中 `payment`、`coupon`、`point`、`account`、`merchant` 包之间存在直接互相访问数据对象和调用内部方法的现象。

1. 从耦合和内聚角度分析问题。
2. 指出违反的设计原则。
3. 给出包结构和接口层面的改进方案。

2.7 七、设计模式题（10 分）

业务中存在变化点：新增 S 级会员和新的积分规则。请说明使用 `Strategy` 如何设计。

要求：

1. 写出模式适用原因。
2. 画出类图。
3. 说明使用到的设计原则。
4. 说明优缺点。

2.8 八、软件构造题（10 分）

某同学把 `payment method selected by switch and membership level type code` 写在一个方法中，方法包含大量条件分支、魔法数字和对数据访问对象的直接调用。

1. 从软件构造角度列出问题。
2. 给出重构方向。
3. 写出一段改进后的代码骨架。

2.9 九、测试题（10 分）

针对 扫码付款与积分计算的核心方法：

1. 说明如何计算圈复杂度。
2. 设计满足 黑盒测试的测试用例集。
3. 补充一个异常路径测试用例。

2.10 十、人机交互题（10 分）

针对 付款码页面、金额确认、密码输入、支付结果反馈，从 HCI 原则角度分析优点、不足和改进建议。至少写 5 点。

3 B 详解

3.1 一、简答题详解

- 软件工程：把系统化、规范化、可量化的方法应用于软件开发、运行和维护，并研究这些方法的学科。展开时补充需求、设计、构造、测试、维护、配置管理、质量保障和项目管理。
- 生命周期：软件从提出、开发、运行、维护到退役的全过程。模型可写瀑布、增量迭代、演化、螺旋。
- SCM：配置项识别、版本管理、变更控制、配置审计、状态报告、发布管理。变更控制为提交请求、评估影响、批准或拒绝、实施、验证、更新状态。
- CI：频繁集成主干，自动构建、测试和质量检查，降低集成风险，缩短反馈周期。
- 再工程：理解旧系统后进行重构、迁移或改造，使系统更易维护或适配新环境。
- 增量迭代与演化：前者按功能批次交付，后者根据反馈持续改变系统形态。

3.2 二、需求题详解

用例示例：

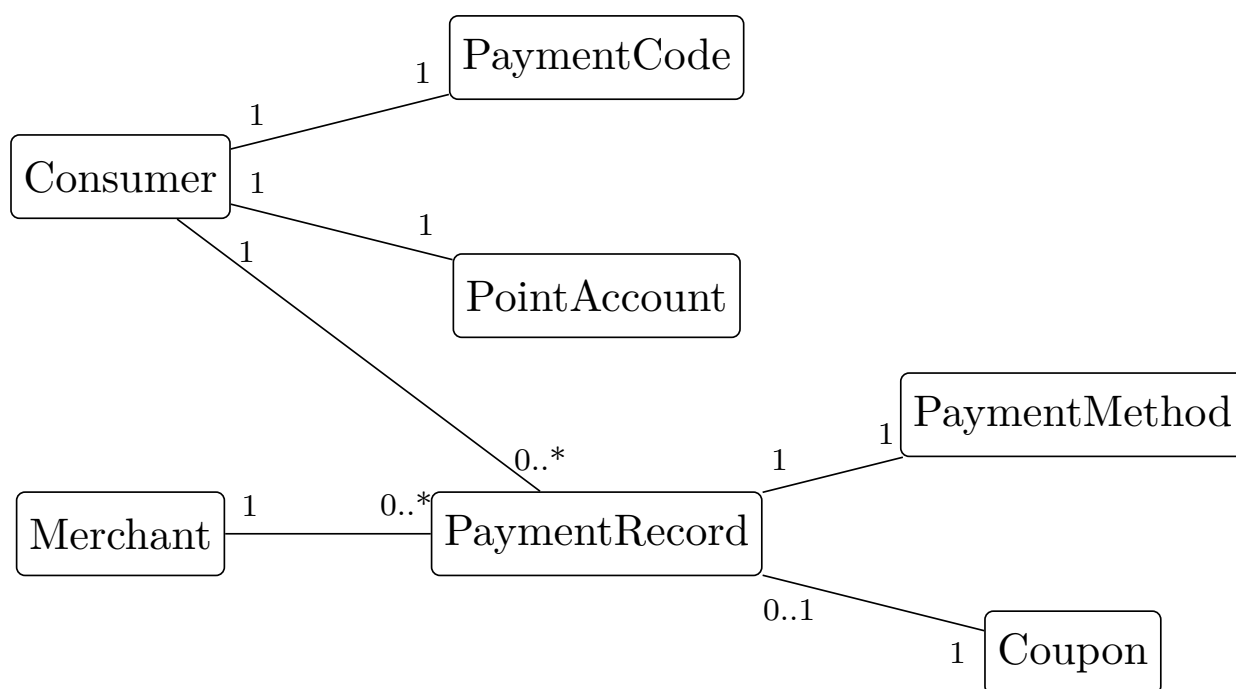
- 消费者扫描商家二维码并输入金额，系统校验付款方式，支付成功后生成付款记录。
- 系统根据会员等级和支付金额计算积分并更新积分账户。

需求层次：

- 业务需求：建设移动支付积分系统，提升扫码付款与积分计算的处理效率，降低人工处理成本。
- 用户需求：消费者希望通过系统自助完成扫码付款与积分计算并查看处理结果。
- 系统级需求：系统接收请求后应校验用户权限、业务对象状态和输入数据，处理成功后保存记录并返回结果。
- 对外 API 属于对外接口需求，因为它规定本系统与外部系统之间的调用方式、输入输出、数据格式和异常处理。

3.3 三、需求分析题详解

概念类图答案：



关键属性：

```

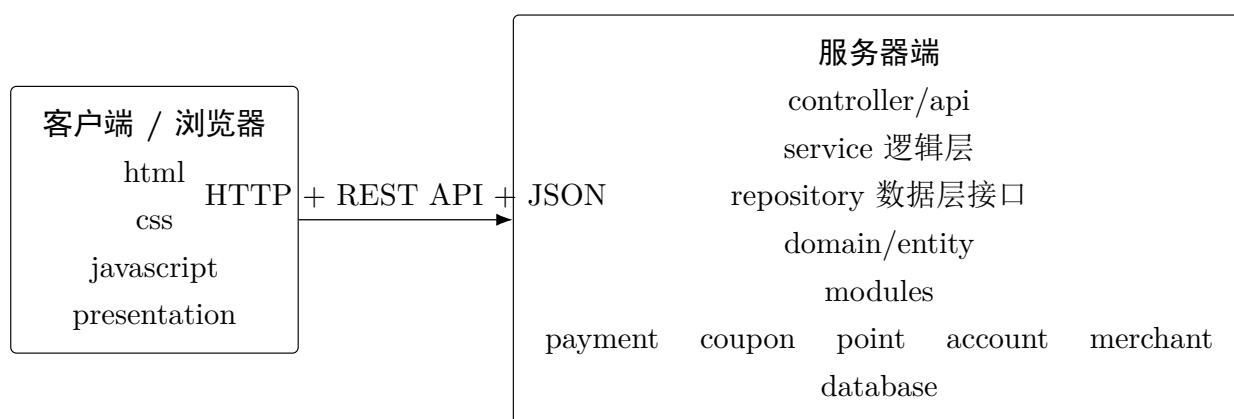
Consumer(userId, name, level)
Merchant(merchantId, name)
PaymentRecord(recordNo, amount, status, paidAt)
Coupon(couponId, type, value)
PointAccount(balance, level)

```

评分点：概念类来自问题域，不写 Controller、Service、Repository；关系写多重性；属性只写业务状态和关键标识。

3.4 四、体系结构设计题详解

物理包图：



接口代码：

```

package edu.nju.pay.service;

public interface PaymentService {
    PayResult pay(PayCommand command);
}

package edu.nju.pay.repository;

public interface PaymentRecordRepository {
    void save(PaymentRecord record);
    List<PaymentRecord> findByConsumerId(Long consumerId);
}

package edu.nju.pay.repository;

public interface PointAccountRepository {
    Optional<PointAccount> findByConsumerId(Long consumerId);
    void save(PointAccount account);
}

package edu.nju.pay.controller;

```

```

@RestController
@RequestMapping("/api/pay")
public class PaymentController {
    private final PaymentService service;

    public PaymentController(PaymentService service) {
        this.service = service;
    }

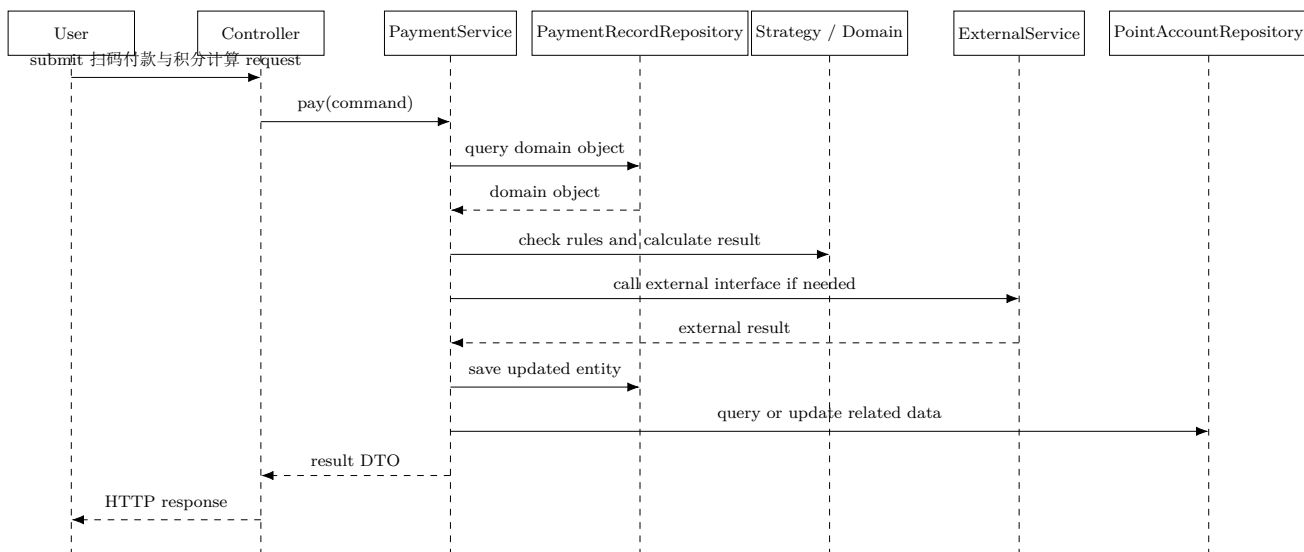
    @PostMapping
    public PayResult submit(@RequestBody PayCommand command) {
        return service.pay(command);
    }
}

```

评分点：Controller 通过构造器注入 Service；Service 和 Repository 都是接口；Controller 不直接访问 Repository。

3.5 五、详细设计题详解

顺序图：



控制风格：

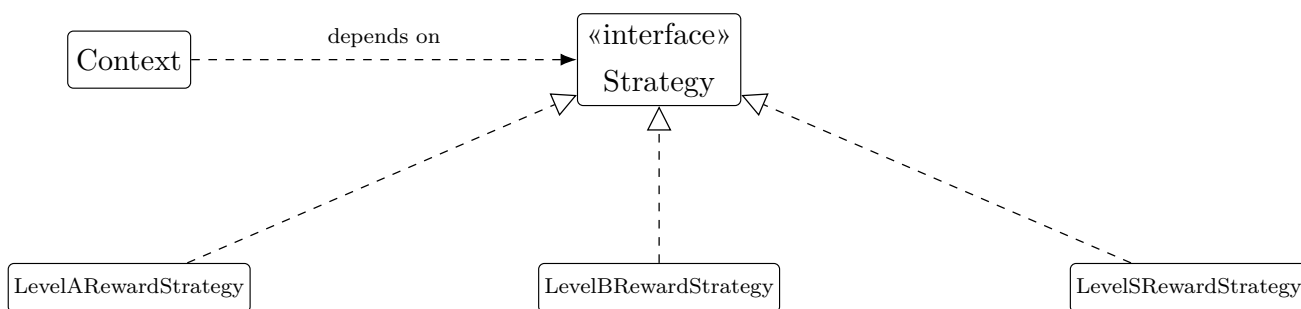
- 集中式控制：少数控制对象掌握流程，流程集中但容易形成大控制器。
- 分散式控制：多个对象共同推进流程，单对象职责轻，但控制流难追踪。
- 委托式控制：入口对象保留主流程，把专业子任务委托给 Service、领域对象、策略对象和 Repository。
- 扫码付款与积分计算适合委托式控制，因为它包含校验、规则计算、外部接口、持久化和结果返回。

3.6 六、模块化与信息隐藏题详解

- 直接互相访问内部对象会形成高访问耦合，修改一个包的数据结构会影响另一个包。
- 业务包之间循环依赖违反低耦合、信息隐藏和 DIP。
- 把跨包交互改成接口调用：上层依赖 Service 接口，Service 依赖 Repository 接口，包内隐藏实现类。
- 把共享数据对象转成 DTO 或领域对象，不暴露内部集合和可变字段。
- 把变化规则封装到策略对象或领域服务中，提高内聚。

3.7 七、设计模式题详解

Strategy 类图：



场景：会员等级积分计算

做法：把变化算法抽成 **Strategy** 接口，**Context** 只依赖接口。

原则：OCP、DIP、SRP。

优点：新增规则时新增实现类，不修改主流程。

代价：类数量增加，需要管理策略选择。

3.8 八、软件构造题详解

问题：

1. 命名不能表达业务含义。
2. 多个业务规则写在一个长条件或 **switch** 中。
3. 魔法数字散落在代码中。
4. 缺少非法输入处理。
5. 方法同时承担校验、计算、持久化或输出职责。

改进：

1. 使用有意义命名。
2. 提取小方法。
3. 用 **Strategy** 或表驱动隔离变化规则。
4. 用 **guard clause** 处理非法输入。
5. 为核心规则补充单元测试。


```
public boolean check(Request request) {  
    validate(request);  
    return rules.stream().allMatch(rule -> rule.pass(request));  
}
```

3.9 九、测试题详解

- T1 正常输入：覆盖成功路径，预期成功。
- T2 关键字段为空：覆盖输入校验失败，预期拒绝并提示。
- T3 对象状态非法：覆盖业务状态失败，预期拒绝。
- T4 外部接口失败：覆盖异常路径，预期回滚或返回失败。
- T5 边界输入：覆盖金额、数量、时间段或容量边界，预期与规则一致。

图复杂度： $V(G) = \text{判定节点数} + 1$ 。

若题目给出 4 个 `if` 或 `while` 判定，则 $V(G)=5$ 。

黑盒测试 作答时写明每个判定的真值和假值如何被覆盖。

3.10 十、人机交互题详解

- 系统状态可见：付款码页面、金额确认、密码输入、支付结果反馈中的提交、处理中、成功、失败状态都要明确显示。
- 一致性和标准：按钮位置、颜色、图标、术语在同类页面中保持一致。
- 错误预防：危险操作要二次确认，非法输入在提交前提示。
- 识别优于回忆：常用选项、历史记录、默认值和示例能降低记忆负担。
- 用户控制和自由：提供返回、取消、撤销或重新提交路径。
- 美学和极简：高频任务放在主路径，次要信息折叠，避免过密表单。

4 C 复盘清单

做完本卷后回看：需求三层次、Web 物理包图、接口代码、委托式控制、设计原则、Strategy、构造重构、黑盒测试、HCI 十项启发式。